

TAFWELA

Problem Statement, Objectives & Software Architecture

DPEI | 2026

1. Problem Statement

Egypt faces a persistent fuel station congestion problem that affects millions of drivers daily. During peak hours and fuel shortage periods, drivers are forced to visit multiple stations before finding one with a manageable queue. This results in significant wasted time, unnecessary fuel consumption from driving between stations, and increased traffic congestion around fuel station areas.

The core issue is a lack of real-time, accessible information about station congestion levels. Drivers currently have no digital tool to check station status before leaving home or while on the road. Station staff have no efficient mechanism to broadcast availability information to incoming drivers.

Problem Dimension	Impact
Time Waste	Drivers spend 20–45 minutes searching for an uncrowded fuel station during peak times.
Fuel Waste	Unnecessary driving between stations increases fuel consumption and vehicle emissions.
Traffic Congestion	Vehicles queuing at or near stations block traffic flow in surrounding streets.
Driver Frustration	Uncertainty about station availability leads to stress and poor user experience.
No Digital Solution	No existing widely-used app in Egypt addresses real-time fuel station congestion visibility.

2. Proposed Solution

TAFWELA (تفويلة) is a Flutter-based mobile application that bridges the information gap between fuel station staff and Egyptian drivers. The app provides:

- A real-time interactive map showing nearby fuel stations with color-coded congestion markers.
- A dedicated Staff Panel allowing station employees to update congestion status instantly.
- GPS-based automatic location detection to show the most relevant nearby stations.
- One-tap navigation via Google Maps integration to guide drivers to their chosen station.
- Search, filter, and favourites features for a personalized and efficient experience.

3. Project Objectives

#	Objective	Success Criteria
O1	Provide real-time congestion visibility for fuel stations	Status updates reflected on driver map within 1 second of staff input
O2	Enable GPS-based discovery of nearby stations	App detects user location and displays nearby stations within 3 seconds
O3	Offer instant navigation to selected stations	Navigate button launches Google Maps with correct destination 100% of the time
O4	Empower station staff to broadcast availability	Staff can update status in under 10 seconds from opening the app
O5	Deliver an intuitive, accessible mobile UI	Onboarding completable in under 30 seconds; usability tested with 5+ users
O6	Build a scalable, maintainable Flutter codebase	Clean Architecture implemented; unit test coverage $\geq 60\%$
O7	Complete and submit all DPEI documentation on time	All 7 sections submitted before their respective deadlines

4. System Capabilities Summary

4.1 Functional Capabilities

- User registration and login (Driver & Staff roles) via Firebase Authentication
- Real-time map with color-coded station markers using Google Maps Flutter SDK
- GPS-based automatic detection of user location via Geolocator
- Station details screen with congestion badge, address, distance, and working hours
- One-tap Google Maps navigation via `url_launcher` deep link
- Staff Panel with congestion update (Low / Medium / High / No Fuel) and history log
- Real-time Firestore listener for sub-second status propagation to all clients
- Search by station name/area and filter by congestion status
- List View as an alternative to the map for comparing station options

- Favourites saved locally via shared_preferences

4.2 System Constraints

- Platform: Android 6.0+ (API 23+); iOS support planned but not primary target for v1
- Backend: Firebase free tier (Spark Plan) — quota monitoring required
- Maps: Google Maps API — billing alerts enabled to prevent service interruption
- Connectivity: Real-time features require active internet; offline mode shows cached data only
- Authentication: Staff accounts created manually by admin — no self-registration for staff

5. Software Architecture

5.1 Architecture Style: Clean Architecture

TAFWELA follows Flutter Clean Architecture, which separates the application into three independent layers. This ensures testability, maintainability, and scalability across the 6-member development team.

Layer	Responsibility	Contents
Presentation Layer	UI rendering and user interaction	Flutter Widgets, Screens, GetX Controllers
Domain Layer	Business logic and use cases	Use Case classes, Entity models, Repository interfaces
Data Layer	Data access and external services	Repository implementations, Firebase datasources, API clients

5.2 Architecture Diagram Description

The system is composed of the following high-level components and their interactions:

Component	Technology	Description
Flutter Mobile App	Flutter / Dart	Cross-platform mobile frontend for both Driver and Staff users. Contains all UI screens and state management logic.
State Management	GetX	Manages application state reactively. Connects UI layer to domain use cases.
Firebase Authentication	Firebase Auth	Handles user registration, login, session persistence, and role-based access control.
Firestore Database	Cloud Firestore	NoSQL real-time database storing station data, user data, staff accounts, and congestion history.
Real-time Listener	Firestore Streams	Pushes live congestion updates from Firestore to all connected Flutter clients within 1 second.
Google Maps SDK	Google Maps Flutter	Renders interactive map, displays custom station markers, and handles GPS-based location.
Geolocator	Geolocator Package	Accesses device GPS to detect driver's current coordinates and calculate distances to stations.
Navigation Service	url_launcher	Launches Google Maps app with deep link for turn-by-turn navigation to selected station.
Local Storage	shared_preferences/SQL lite	Persists user favourites and onboarding state locally on the device.
Cloud Functions	Firebase Cloud Functions	Optional: Sends push notifications when a station's status changes to 'No Fuel'.

5.3 Folder Structure

The Flutter project follows a feature-first Clean Architecture folder structure:

Folder Path	Contents
lib/core/	Shared utilities, constants, theme, error handling, network checker
lib/features/auth/	Login, Register screens + Auth use cases + Firebase Auth datasource
lib/features/map/	Map screen, station markers, GPS logic + Firestore station datasource
lib/features/station_details/	Station details screen, navigation button, favourites logic
lib/features/staff_panel/	Staff login, dashboard, status update, history log
lib/features/search/	Search bar, filter logic, list view screen
lib/features/onboarding/	Onboarding screens, skip logic, first-launch detection
lib/injection_container.dart	Dependency injection setup (GetIt)
test/	Unit tests, widget tests organized by feature

5.4 Data Flow Overview

The following describes the high-level data flow for the two primary use cases:

Driver: View Real-time Station Map

Step	Action
1	App launches → Geolocator detects driver GPS coordinates.
2	Map screen queries Firestore for stations within the configured radius.
3	Firestore returns station documents → Flutter renders custom colored markers on Google Maps.
4	Firestore real-time listener remains active → any staff update triggers immediate marker color change.
5	Driver taps marker → Station Details screen loads from local Firestore snapshot.
6	Driver taps Navigate → url_launcher opens Google Maps with station coordinates.

Staff: Update Congestion Status

Step	Action
1	Staff member opens app → Firebase Auth verifies staff role.
2	Staff Dashboard loads station name, current status, and last update timestamp from Firestore.
3	Staff selects new congestion level → Confirmation dialog displayed.
4	Staff confirms → App writes new status + staff ID + timestamp to Firestore.
5	Firestore triggers real-time update → all connected driver clients receive new status within 1 second.
6	Change is recorded in station history log in Firestore.